

Open in app ↗

Sign up

Sign in

Medium

Search



Android



Android App Development



AndroidDev



C Programming



Cplusplus



The Android Architecture [feat. Native C/C++ Libraries & Android Runtime]

6 min read · Mar 18, 2025



Emmanuel Mutura™

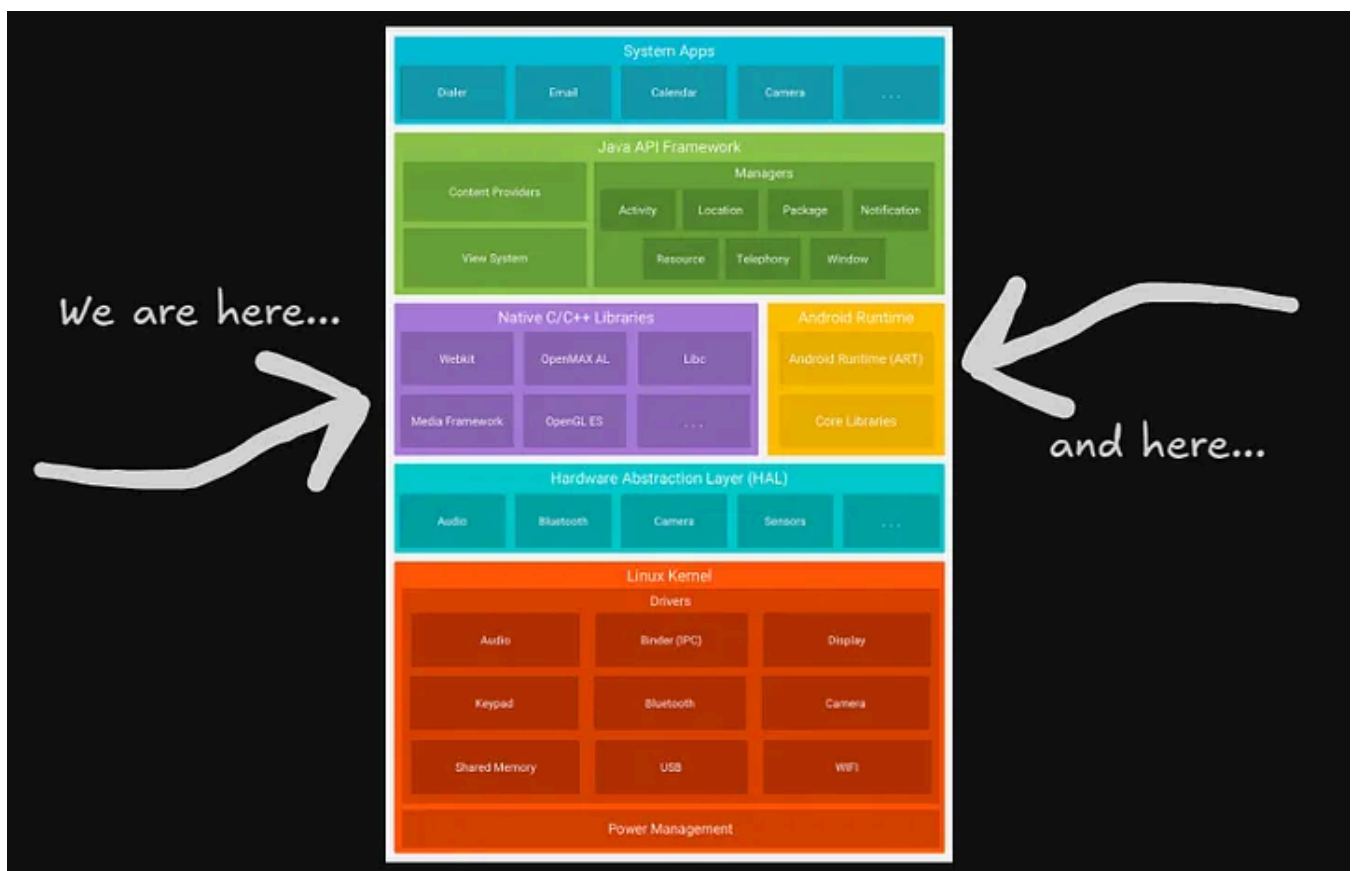
Follow



Listen



Share



They say that “Third time’s a charm”. While I am still not sure who “they” refers to, I agree with them because today’s article is a special one as it talks about not 1 but 2 components of The Android Architecture. Previously, we learnt that The Hardware Abstraction Layer [HAL] provides the rest of Android access to The Linux Kernel,

which is the core of the OS. Now, have you ever wondered what the building blocks of these components are? Also, have you asked yourself how all those Android apps can run on a single device without instantly depleting its battery level due to their unique performance requirements. If that is you, then this is your article for it talks about **The NativeC/C++ Libraries** and **The Android Runtime**. Again, the Car analogy will be used for illustration purposes...

1. Overview: What Are They?

The Native C/C++ Libraries are a set of core, low-level libraries written in C and C++ that power performance-critical tasks — such as graphics rendering, media processing, and hardware access — within Android...

In contrast, the Android Runtime [ART] is the managed runtime environment that executes Android applications. ART replaces Dalvik with Ahead-Of-Time [AOT] and Just-In-Time [JIT] compilation techniques, advanced garbage collection, and improved memory management. Together, these components form the backbone of Android's performance: the native libraries provide near-hardware-level speed and efficiency, while ART manages application execution with smart optimizations...

Analogy

Think of the Native C/C++ Libraries as the car's finely engineered engine and essential mechanical components that deliver raw horsepower. ART, on the other hand, is like the sophisticated onboard computer system that monitors, optimizes, and adapts engine performance in real time. Together, they ensure your vehicle not only has the power to accelerate but also runs smoothly and efficiently...

2. Performance and Efficiency: Raw Power and Optimised Control

Native libraries are compiled to machine code, allowing them to run with minimal overhead and at near-hardware speeds. This makes them ideal for tasks that demand high performance, such as 3D graphics or real-time media processing. ART enhances this performance by compiling apps Ahead-Of-Time [AOT] for fast startup and using Just-In-Time [JIT] compilation to optimise frequently executed code paths during runtime. Additionally, ART's efficient garbage collection ensures that memory is reclaimed promptly, keeping the system responsive even under heavy workloads...

Analogy

Imagine the native libraries as a high-performance engine built for raw power, capable of delivering maximum horsepower. ART is the finely tuned transmission and Engine Control Unit [ECU] that adjusts gear ratios on the fly, ensuring that the raw power is harnessed smoothly and efficiently. Together, they provide both the muscle and the smart control that let your car accelerate quickly and maintain peak performance under various driving conditions...

3. Standardised APIs and Integration: Consistency Across the Board

The Android Native Development Kit [NDK] provides a consistent set of APIs that allow developers to harness the power of native libraries, ensuring that performance-critical code works uniformly across different hardware. ART leverages the J*va Native Interface [JNI] to bridge the gap between managed Java/Kotlin code and native C/C++ code, enabling seamless interoperability. This standardised approach minimizes fragmentation, ensures compatibility, and allows both layers to work in concert without the developer needing to manage low-level hardware specifics...

Analogy

Picture a car assembly line where every vehicle is built with standardised components — engines, brakes, and electronics — that ensure consistent performance across models. The integration via JNI is like a finely engineered gear shift that smoothly transitions power from the engine to the drivetrain, ensuring the car operates reliably regardless of individual parts. In this way, both the native libraries and ART work together under a unified standard, much like standardized car parts ensure a dependable driving experience...

4. Graphics, Media, and Real-Time Operations: Handling the High-Octane Tasks

Native libraries, including OpenGL ES, Vulkan, and media frameworks like `libstagefright`, are optimized for intensive graphics rendering and media processing tasks. They provide the raw computational power necessary for high-

quality visuals and real-time audio/video playback. ART complements these capabilities by ensuring that the application code, which may invoke these libraries frequently, runs with minimal overhead through dynamic optimizations. This cooperation results in a smooth, responsive user experience even during resource-intensive operations...

Get Emmanuel Mutura™'s stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

Analogy

Think of the native libraries as a turbocharged engine that delivers instantaneous power for rapid acceleration and dynamic performance — ideal for high-speed racing. ART acts like an adaptive transmission that seamlessly shifts gears based on driving conditions, ensuring that every burst of power is effectively translated into smooth motion. Combined, they create a driving experience where raw performance and refined control work together to handle even the most demanding scenarios...

5. Memory Management and Garbage Collection: Sustaining the Drive

Native C/C++ code is designed for direct and efficient memory management, often requiring the developer to handle allocation and deallocation with care. ART introduces an advanced garbage collection mechanism that automates memory reclamation, reducing the risk of leaks and latency issues. This synergy ensures that while native libraries provide the power through efficient, low-level memory operations, ART keeps the system's memory clean and well-organized, maintaining high performance and stability over prolonged use...

Analogy

Imagine the native memory management as the car's precise fuel injection system that delivers just the right amount of fuel to each engine component, ensuring optimal performance. ART's garbage collection is akin to an automated maintenance system that continuously cleans and adjusts the engine, preventing clogs and wear. Together, they keep your car running at peak efficiency — always ready for the next drive without unexpected breakdowns...

6. Debugging, Profiling, and Optimisation: Tuning the Machine

A robust ecosystem of tools — such as systrace, gdb, and performance profilers — is available to diagnose and optimize both native libraries and ART's execution. Developers can profile low-level C/C++ code for bottlenecks, memory leaks, or inefficient loops, while ART's diagnostic tools provide insights into runtime behavior, garbage collection events, and JIT performance. This comprehensive approach to debugging and optimization ensures that applications are not only high-performing but also reliable and resource-efficient under diverse conditions...

Analogy

Think of it as having a state-of-the-art onboard diagnostic system in your sports car that continuously monitors engine performance, fuel efficiency, and mechanical integrity. Advanced diagnostic tools help mechanics fine-tune the engine and transmission, ensuring peak performance. Similarly, the debugging and profiling tools for native libraries and ART work together to “tune” your Android device, ensuring that every component operates in harmony for the ultimate driving experience...

7. Evolution and Future Trends: Driving Innovation Forward

Both the Native C/C++ Libraries and ART are evolving rapidly to meet the demands of next-generation Android applications. Modern C/C++ standards (like C++11 and beyond) and improved compiler optimisations continue to enhance native performance, while ART is integrating predictive optimizations, machine learning techniques, and more efficient memory management strategies. These

advancements pave the way for new hardware innovations, better energy efficiency, and more secure and responsive applications — ensuring that Android remains at the cutting edge of mobile technology...

Analogy

Consider this the evolution of automotive technology — from traditional internal combustion engines to smart, hybrid, and electric vehicles equipped with advanced driver-assistance systems. Just as modern cars are continually upgraded with innovations to enhance safety, performance, and efficiency, the native libraries and ART are being reengineered to deliver a futuristic driving experience. Together, they ensure that your Android “vehicle” stays ahead of the curve, ready to tackle tomorrow’s challenges with state-of-the-art technology...

UP NEXT

The J*va API Framework: <https://medium.com/@emmanuelmutura/74fb865f6431>

CONCLUSION

Apart from the desire to drive your Car right now, I hope that you have now gotten a better and more technical understanding of The Native C/C++ Libraries and Android Runtime [ART] in the context [pun intended] of The Android Architecture. Stay tuned for both a higher-level summary of this topic as well as the next layer of The Android Architecture. Until then, I hope that you remember what you learned today each time you go for your road trip :)...



REFERENCES

Platform Architecture. (2023, May 4). Android Developers.
<https://developer.android.com/guide/platform>